CodeA11y - Leveraging Large Language Models for Runtime-Aware, Governed, and Guardrail-Verified Accessibility Implementation

Rachit Chandak
Future of Accessibility, TCS Research and Innovation
rachit.chandak@tcs.com

Sumeet Agrawal
Future of Accessibility, TCS Research and Innovation
sumeet.a@tcs.com

Charudatta Jadhav
Future of Accessibility, TCS Research and Innovation
charudatta.jadhav@tcs.com

Manjira Sinha
Data and Decision Sciences, TCS Research and Innovation
sinha.manjira@tcs.com

***Abstract*—Despite the existence of the Web Content Accessibility Guidelines (WCAG)[1] and the availability of various automated accessibility linters, a significant number of web applications continue to exhibit accessibility issues such as broken keyboard navigation and poor screen reader usage, ultimately rendering web applications inaccessible to people with disabilities[2]. This paper introduces CodeA11y, an AI-powered accessibility platform designed to transform accessibility from post-hoc compliance into a continuous, engineering-led capability across the software development lifecycle (SDLC). While CodeA11y spans the full SDLC, this paper focuses on its development-phase agent responsible for source-level remediation. By leveraging Large Language Models (LLMs), CodeA11y detects, explains, and resolves accessibility violations by analyzing source code and applying WCAG-guided reasoning to generate code-level fixes during development. Beyond static and structural analysis, we extend the methodology along three new dimensions that directly target the limitations of source-only remediation: (i) a runtime applicability layer that instruments and renders the live application to ground each source file in the WCAG checks that actually apply to its rendered artifacts; (ii) an output-verification guardrail that scores every LLM-generated rationale against an evidence base of official WCAG failure conditions and sufficient techniques, rejecting hallucinated criteria and flagging weakly grounded reasoning for review; and (iii) a governance and evidence layer that enforces per-user audit/fix budgets and produces an audit-ready trail of every detection, fix, and review decision. The system was evaluated against a human-validated ground-truth set of 150 accessibility scenarios aligned with WCAG 2.x Level A and AA success criteria (SCs)[3], grouped into three categories: static, structural, and behavioral. The results demonstrate that CodeA11y achieves high remediation coverage for static violations (98.11%) and structural violations (90.16%) at the source-code level, while behavioral violations remain more challenging (66.67%), reflecting the additional difficulty of runtime interactions between users and assistive technologies. The runtime-aware, guardrail-verified, and governed extensions described here are our principal response to that gap, narrowing the distance between source-level edits and the dynamic behavior they ultimately govern.**

***Index Terms*—Web Accessibility, LLM driven accessibility solutions, WCAG compliance, Accessibility implementation, Runtime instrumentation, LLM guardrails, Governance**

I. Introduction

Accessibility is one of the core requirements for software and web systems, ensuring that everyone, regardless of their abilities, can access information and services equally. In this regard, WCAG have been accepted as the benchmark standards. Governments worldwide have adopted these into laws, for example: i) EU's EN 301 549[4], ii) US DOJ's Title II requiring WCAG 2.1 Level AA compliance by 2026–2027[5,6], and iii) India's IS 17802[7]. Despite these guidelines and the existence of automated linters, developers still struggle in practice to understand how assistive technologies behave. While some tools can detect certain violations automatically, fixing them is still a largely manual, time-consuming process. On the other hand, recent advances in generative AI driven, LLM-assisted coding have resulted in significant productivity increases, a jump of 55.8% faster task completion according to [1]. However, their application to accessibility has largely been limited to detecting issues [2], [3] with minimal research in automated remediation [4].

In this paper, we present CodeA11y[8], an LLM-based[9] developer-facing tool that detects, explains, and remediates WCAG violations at the project source-code level. Unlike traditional linters that analyze files in isolation, CodeA11y

[1]https://www.w3.org/WAI/standards-guidelines/wcag/

[2]https://webaim.org/projects/million

[3]https://www.allaccessible.org/blog/wcag-level-a-aa-and-aaa-whats-the-difference

[4]https://www.etsi.org/human-factors-accessibility/en-301-549-v3-the-harmonized-european-standard-for-ict-accessibility

[5]https://www.federalregister.gov/documents/2024/04/24/2024-07758/nondiscrimination-on-the-basis-of-disability-accessibility-of-web-information-and-services-of-state

[6]https://www.ada.gov/resources/2024-03-08-web-rule

[7]https://www.barrierbreak.com/indias-digital-inclusion-initiative-is-now-a-law-is-17802-mandates-accessible-ict-products-and-services

[8]Find demo with voiceover

[9]Our demo incorporates GPT-4o.

constructs a dependency graph across source files, stylesheets, and scripts to perform context-aware analysis and generate coordinated fixes. Crucially, CodeA11y does not stop at static source analysis: it renders the running application to determine which WCAG checks are actually *applicable* to each file, verifies every model-generated rationale with a multi-layer guardrail before it reaches the developer, and governs the entire process under enforceable budgets and an audit-ready evidence trail. We evaluate CodeA11y across two critical dimensions for each of three categories of violations—static, structural, and behavioral:

1) **Identification Coverage:** How comprehensively does CodeA11y identify issues across different WCAG violations.
2) **Remediation Coverage:** How effectively does CodeA11y generate fixes across different WCAG violations.

A. Motivation & Background

Despite mature WCAG standards and regulatory mandates, organizations continue to experience challenges (often connected) in developing accessible products, such as:

- **Developer's knowledge gap:** Accessibility training is uneven across the industry. Developers frequently lack practical knowledge of when to use native semantics versus ARIA patterns, how to manage focus correctly, and how assistive technologies interpret dynamic UI changes. This gap pushes teams toward surface-level fixes rather than truly inclusive implementations [5].
- **Framework heterogeneity:** Large enterprises build experiences across heterogeneous front-end stacks (e.g., React, Angular, Vue, native iOS and Android) where the same UI intent can be implemented in completely different ways. WCAG requirements are platform-agnostic, but accessibility implementations are inherently stack-specific, making fixes non-portable and increasing the developer burden significantly.
- **Limited automation:** Current automated remediation tools remain limited in scope. They can detect many syntax-level violations, but accessibility fixes are not one-size-fits-all. Static fixes (e.g., adding alt text) are straightforward, but structural changes (replacing non-semantic elements, refactoring forms) affect layout, event handling, and application logic. Behavioral fixes (managing focus, supporting live announcements) require runtime validation across browsers and assistive technologies [6].
- **Scale and Complexity:** Enterprise applications contain thousands of UI components and millions of lines of code, accumulating significant accessibility debt across templates, stylesheets, and component libraries. Manual remediation and postponing fixes increase remediation cost and complexity [7]. Organizations face a substantial financial burden beyond remediation costs—71% of disabled users abandon inaccessible websites[10], representing $13 trillion in lost purchasing power[11] and accessibility lawsuits average $5,000 to $20,000 in settlements[12].
- **Trust and accountability:** Because LLMs can produce fluent but unfounded rationales, AI-generated accessibility advice can be confidently wrong—citing success criteria that do not apply or fabricating technique identifiers. Without verification and an audit trail, such output is unsafe to apply at enterprise scale and indefensible under regulatory scrutiny.

These factors transform accessibility from just a technical concern into a strategic business imperative affecting revenue, risk exposure, and market access. Existing linters cannot infer developer intent, user-interaction context, or the semantic meaning behind code choices. They flag issues but leave remediation largely manual. We built CodeA11y to address this gap by bridging two paradigms: Gen-AI-powered, LLM-assisted reasoning and the development of default-accessible solutions for inclusion and impact.

B. Technical Novelties

Although several technical approaches exist in the literature for WCAG-related identification and remediation, several limitations remain [3], [4], [8]–[11]. Many proposed approaches operate on rendered HTML or DOM snapshots, analyzing accessibility after the interface is generated [12]. Additionally, proposed solutions are often implemented as isolated scripts or page-level prototypes and are not integrated into developer workflows such as IDEs, code-review pipelines, or multi-file project structures. As a result, they struggle to address accessibility issues in applications where behavior, semantics, and styling are distributed across multiple components and files. We address these gaps by analyzing accessibility directly at the source-code level while grounding that analysis in runtime evidence.

CodeA11y is a human-in-the-loop, agentic AI platform that transforms accessibility from a post-hoc compliance activity into a continuous, engineering-led capability embedded across the SDLC. Instead of stopping at detection, CodeA11y operationalizes accessibility through a **Generate** $\rightarrow$ **Verify** $\rightarrow$ **Evidence** model, as explained in Section II. In essence it works as a multi-agent accessibility framework, spanning:

i. Design-phase agents that identify accessibility risks early at the prototype and specification stage.

ii. Development-phase agents that generate explainable, WCAG-mapped source-code fixes with diff-based review and rollback.

iii. Testing-phase agents that validate fixes and prevent regressions through CI/CD integration.

iv. Governance and evidence agents that produce continuous compliance reports aligned to WCAG regulations and laws.

[10]https://www.clickawaypound.com/cap16finalreport.html

[11]https://www.weforum.org/stories/2023/12/driving-disability-inclusion-is-more-than-a-moral-imperative-it-s-a-business-one/

[12]https://reciteme.com/us/news/ada-web-compliance-lawsuits/

This paper contributes three methodological extensions to the development-phase agent that, to our knowledge, have not been combined in prior LLM-based accessibility remediation systems:

1) A **Runtime Applicability layer** that instruments source files, renders the live application, and maps rendered DOM artifacts back to their originating source lines, so that the audit is grounded in the WCAG checks that genuinely apply to each file at runtime rather than in static guesses.
2) An **Output-Verification Guardrail** that treats every LLM rationale as a claim to be falsified, scoring it across six independent signals against an evidence base of official W3C WCAG failure conditions and sufficient techniques, and gating it into accept/review/reject decisions.
3) A **Governance and Evidence layer** that enforces per-user audit and fix budgets, persists normalized project snapshots, and records an audit-ready trail of detections, fixes, and human decisions.

II. System and Architecture

CodeA11y combines heuristic code analysis, runtime instrumentation, and Large Language Model (LLM) reasoning to detect accessibility violations, explain their impact, generate context-aware remediation code, and verify that remediation before it reaches a developer. As shown in Figure 1, the architecture follows a multi-layered, agent-oriented design consisting of four primary layers: a Client Layer, a Processing Layer, a Service Layer, and a Data Layer. This layered architecture enables scalable analysis of large code-bases by decomposing the analysis process into smaller units that can be processed in parallel. The methodology is organized around the **Generate** $\rightarrow$ **Verify** $\rightarrow$ **Evidence** model: the Processing Layer *generates* runtime-grounded findings and fixes; the guardrail and human-in-the-loop workflow *verify* them; and the Service and Data layers *govern* the process and produce *evidence*.

A. Client Layer

The Client Layer integrates CodeA11y directly into the developer environment through a Visual Studio Code extension. The extension serves as the primary interface between the developer and the analysis engine. When the developer initiates an accessibility scan, the extension compiles the relevant project files and invokes the AI binary executable, which orchestrates the accessibility analysis pipeline. The extension provides real-time feedback within the IDE by presenting accessibility violations grouped by severity level and associated WCAG success criteria, and it renders a diff-based fix-review surface in which guardrail verdicts (Section II-D) are displayed alongside each proposed edit.

B. Processing Layer

The Processing Layer forms the core intelligence of the CodeA11y system. It employs a multi-agent architecture to coordinate accessibility-analysis tasks across the source code, and it is augmented by a runtime applicability stage that grounds each agent's reasoning in observed application behavior.

*1) **Manager Agent**:* The Manager Agent acts as the central orchestration component responsible for coordinating the analysis pipeline. Its primary responsibilities include:

- Constructing a dependency graph of the project files
- Identifying relationships between components, templates, and stylesheets
- Partitioning the codebase into analysis bundles
- Triggering the runtime applicability preflight (Section II-C) for the scoped files
- Dispatching bundles—enriched with per-file applicability context—to worker agents
- Aggregating results from distributed analysis tasks

Rather than analyzing files independently, the Manager Agent groups tightly coupled code artifacts into bundles, a key feature of CodeA11y that improves contextual understanding during accessibility evaluation.

*2) **Intent Classifier**:* The Intent Classifier analyzes user input to determine the appropriate analysis mode and routes requests accordingly. This module categorizes user intent into three primary modes:

1) *Conversational Query:* the user seeks information, explanations, or guidance about accessibility concepts (e.g., "What is ARIA?", "How do I make a modal accessible?").
2) *Assessment Only:* the user requests an accessibility audit without automated fixes (e.g., "Analyze this component for WCAG violations", "Scan my codebase").
3) *Assessment and Remediation:* the user requests both violation detection and automated fix generation (e.g., "Fix accessibility issues in this file", "Remediate WCAG violations").

Based on this intent classification, the system routes requests to the appropriate agent:

- *Chat Assistant:* handles conversational queries, provides explanations, and offers accessibility guidance without code analysis.
- *Worker Agents:* perform accessibility assessment on code bundles, generating structured violation reports.
- *Worker Agents + Fix Generation:* execute the full assessment and remediation pipeline, producing both violation reports and corrected code.

This classification allows the system to optimize processing resources by invoking only the necessary components for each user request, avoiding unnecessary analysis when users seek informational support and preventing automated fixes when users prefer manual remediation.

*3) **Worker Agents**:* Worker Agents are the core components that perform accessibility analysis on the assigned code bundles. For each code bundle, the agent receives the contextual information required for accessibility analysis, including:

i. Element structure

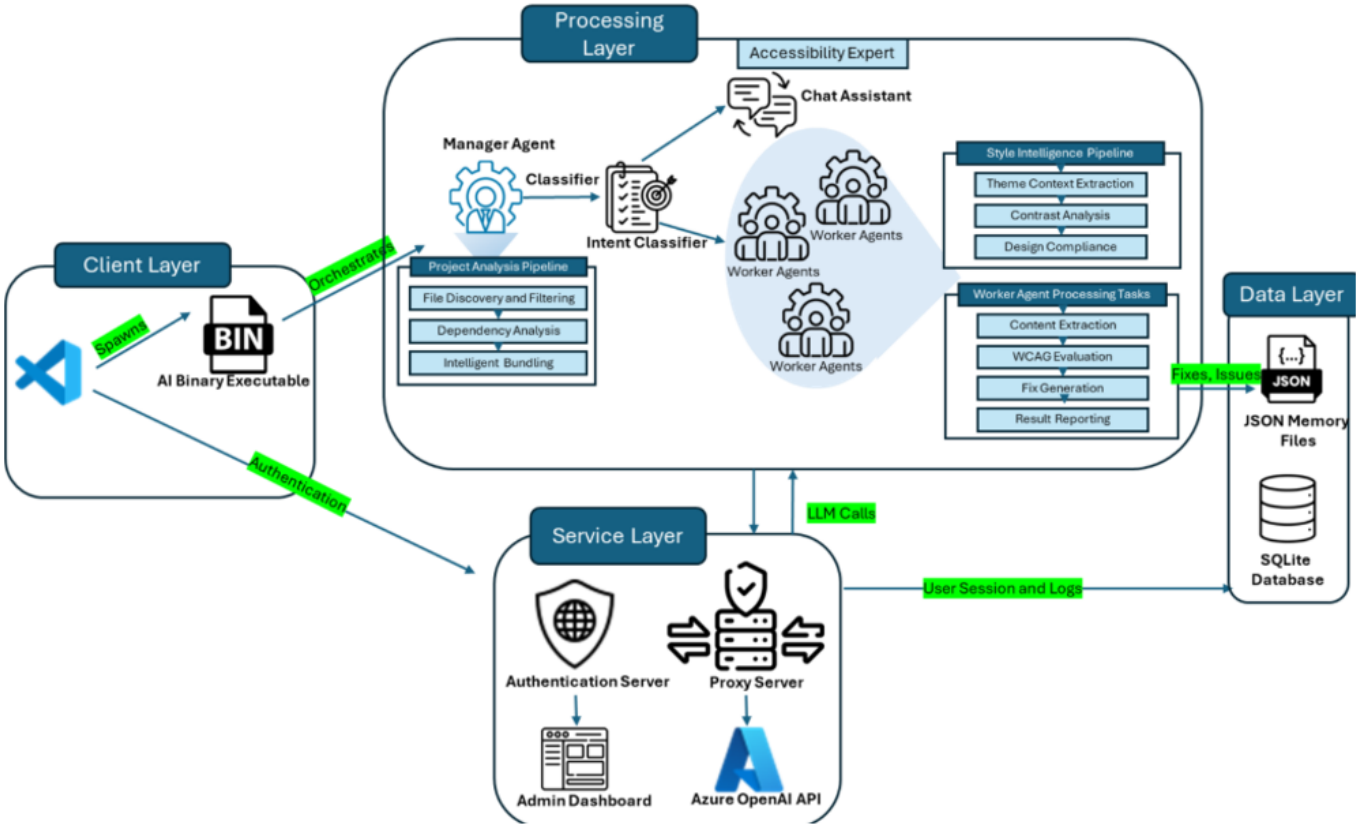

Fig. 1. CodeA11y Architecture

ii. Attributes and ARIA roles
iii. Surrounding text content
iv. Associated stylesheets
v. Component hierarchy
vi. Runtime applicability context (the file-specific list of WCAG checks observed to apply at runtime).
This contextual representation enables the system to analyze accessibility properties that may span multiple source files. Worker Agents submit structured prompts—including the relevant WCAG guidelines, the code snippet, the runtime applicability context, and evaluation instructions—to the LLM to obtain a structured response describing whether a violation exists, an explanation of the issue, and a recommended remediation. When runtime applicability context is present, the agent is instructed to treat it as the primary, file-specific list of relevant WCAG checks and to prioritize the corresponding source-line anchors, auditing additional issues only where the code provides direct evidence.

*4) **Style Intelligence Pipeline**:* The CSS analysis module analyzes global stylesheets to build a comprehensive theme context including color variables, typography scales, spacing systems, and component-specific style relationships. This context enables accurate contrast-ratio calculations and design-system compliance checking. During the runtime stage, this static theme context is complemented by a live contrast analyzer that computes the rendered foreground/background relationships of each detected artifact, allowing color-contrast applicability to be confirmed against what is actually painted rather than against declared styles alone.

*5) **WCAG Compliance Knowledge Base**:* The system incorporates a comprehensive WCAG 2.2 knowledge base that serves as the foundation for consistent accessibility analysis. It is constructed in the following manner:

- **Step 1:** For each element on a webpage, the set of syntax tags is identified.
- **Step 2:** All relevant WCAG 2.2 guidelines / success criteria are identified against the set of tags.
- **Step 3:** Success criteria are organized by conformance level.
- **Step 4:** Rules are constructed to transform the textual descriptions into instructions that can be processed by the LLM prompt.

This knowledge base is directly embedded into the AI analysis and prompts to ensure uniform interpretation of accessibility standards across all code reviews. The same knowledge base, augmented with the official W3C catalog of failure conditions and sufficient techniques, is reused as the evidence base for output verification (Section II-D).

C. Runtime Applicability Layer

A central limitation of source-only analysis is that it cannot distinguish, a priori, which success criteria genuinely apply

to a given file: the same component may render differently—or not at all—depending on routing, conditional logic, and data. CodeA11y addresses this with a Runtime Applicability layer that renders the live application, observes the artifacts each source file actually produces, and maps the applicable WCAG checks back onto exact source lines. This grounds the subsequent LLM audit in observed behavior and is our primary mechanism for narrowing the static–behavioral gap.

The layer is driven by an **applicability catalog**, a curated mapping from artifact archetypes (e.g., `anchor`, `button`, `form`, `image`, `table`, landmarks) to two classes of checks: *static-applicable* checks, which always apply when the artifact is present, and *conditional-applicable* checks, each guarded by a declarative rule (a field/regular-expression predicate over the artifact's selector, text, labels, or DOM signature) and an associated human-readable condition (e.g., "Anchor contains icon-only content", "Link meaning is conveyed by color alone"). This catalog encodes accessibility expertise once and applies it uniformly across heterogeneous stacks.

Given a scoped set of target files from the Manager Agent, the layer executes the procedure summarized in Algorithm 1:

1) **Project resolution.** The project's source directory, start command, and a candidate URL are inferred (or supplied), so the layer can launch the application as the developer would.
2) **Cache check.** A metadata fingerprint over the source directory and target URL is computed; if a fresh applicability result exists for the same scope, it is reused, and the live run is skipped.
3) **Source instrumentation.** A backup of the source tree is taken, and instrumentable files are annotated so that each rendered element carries a `data-source-loc` marker encoding its originating `file:line:column`. This is the bridge that lets us translate runtime observations back into source-level edits.
4) **Live rendering.** The application's dev server is started and the target page is loaded in a headless Chromium instance via Playwright; the page is allowed to reach network idle and settle so that dynamically loaded content is present.
5) **In-page detection.** A contrast analyzer and a registry of artifact detectors are injected into the page. Detectors enumerate interactive widgets and structural components, ignoring framework dev-overlay nodes, and record each artifact's selector hint, bounding rectangle, accessible text/labels, visibility state, source mapping, and rendered contrast findings.
6) **Artifact-to-source mapping.** Each detected artifact is matched against the applicability catalog: static-applicable checks are attached unconditionally, and conditional-applicable checks are attached only when their declarative rule matches the artifact's observed properties. Artifacts that resolve to a source location are aggregated per file.
7) **Evidence capture.** For each mapped artifact, a cropped element screenshot is captured and stored as visual evidence, to be surfaced later in the report and review surfaces.
8) **Context assembly and caching.** The result is assembled into a per-file applicability context—each file listing its applicable checks with criterion, title, source line, artifact identity, condition (if conditional), and any screenshot—summarized (files, artifacts, static vs. conditional check counts), cached, and finally filtered to the originally scoped files. The source tree is restored from backup in all cases.

Algorithm 1 Runtime Applicability Preflight

input: project root P, scoped target files F
output: per-file applicability context C
resolve start command and URL for P
if fresh cache exists for (P, url, F) **then**
  return cached context
end if
back up source tree; instrument files with `data-source-loc`
start dev server; load URL in headless browser; settle
inject contrast analyzer and artifact detectors
$A \leftarrow$ detected widgets and components (with source mapping)
for all artifact $a \in A$ **do**
  attach *static-applicable* checks for a's archetype
  for all conditional check k of a's archetype **do**
    if rule(k) matches observed properties of a **then**
      attach k to a
    end if
  end for
  capture element screenshot for a
end for
aggregate checks per source file into context C
cache C; restore source tree from backup
return C filtered to F

The resulting context is injected into each Worker Agent's prompt as a dedicated RUNTIME APPLICABILITY block listing, for the file under analysis, every applicable check with its source line, originating artifact, optional condition, and screenshot availability. When the preflight yields no file-specific checks (e.g., a file that does not render on the chosen route), the agent is explicitly told to audit only issues directly evidenced by the code and to cite exact line evidence. In this way, runtime applicability acts both as a recall aid—surfacing behavior-dependent checks that static inspection would miss—and as a precision constraint that suppresses speculative findings.

D. Output-Verification Guardrails

LLMs can produce fluent rationales that cite inapplicable success criteria or fabricate technique identifiers. Applying such output unmodified is unsafe at enterprise scale. CodeA11y therefore treats every model-generated rationale as a claim to be *falsified* before it is shown to a developer. The

guardrail is a server-side service that evaluates a (criterion, rationale) pair against an evidence base built from the official W3C WCAG 2.2 catalog—87 success criteria, each annotated with its documented failure conditions and sufficient techniques. At startup the service embeds this catalog into two vector stores (one over failure conditions, one over techniques) so that evaluation is a fast nearest-neighbor and rule-matching operation at request time.

For each rationale, the guardrail computes six independent signals (Figure 1), deliberately combining lexical, semantic, and pragmatic evidence so that no single fragile signal can dominate:

1) **TF–IDF / embedding similarity** between the synonym-expanded rationale and the failure conditions of the claimed criterion.
2) **Keyword/phrase match** measuring overlap with key phrases drawn from those failure conditions.
3) **Sufficient-technique alignment** measuring similarity to the criterion's documented techniques.
4) **Negation awareness**, a scope-aware signal distinguishing language that describes a *failure* from language that describes a *pass*, applied as both a signal and a modifier.
5) **Concept coverage**, the fraction of the criterion's salient concepts present in the rationale.
6) **Specificity**, distinguishing page-specific reasoning with concrete evidence from generic boilerplate.

These signals are combined into a weighted *composite score*, and a *confidence level* (high/medium/low) is derived from how many signals independently agree. Three additional integrity checks run alongside the score: a **fabricated failure-ID validator** that hard-rejects any rationale referencing non-existent WCAG failure or technique identifiers; a **description verifier** that flags references whose descriptions do not match the official definitions; and a **coherence check** that flags internally contradictory reasoning. For medium- and high-confidence results, a **misattribution cross-validator** additionally checks whether the rationale aligns more strongly with a *different* success criterion than the one claimed, flagging likely mislabeling. The composite score, confidence, and integrity checks pass through an enforcement gate that yields one of three actions:

- **Accept** (surfaced to the developer as *accurate*)—the rationale aligns with known failure conditions at adequate confidence.
- **Flag** (*review*)—weak alignment, low agreement, misattribution risk, or a failed integrity check; the developer is advised to review.
- **Reject** (*ignore*)—no detectable alignment, or fabricated identifiers; the rationale is treated as a hallucination.

Each verdict is returned as compact metadata (flag, source action, reason, composite score, confidence, maximum similarity, matched failure condition, and inferred WCAG level) attached to the corresponding issue or fix. In the IDE, fixes are sorted so that *reject*/*review* items rise to the top of the review queue, focusing human attention where the model is least trustworthy. Guardrails are enabled per workspace and fail safe: if the guardrail service is unreachable or a fix lacks a single resolvable WCAG criterion or description, the edit is conservatively marked for review rather than silently accepted. The guardrail is thus a metacognitive verification stage—an automated, evidence-grounded "second opinion" on the model's own reasoning—directly analogous to self-reflective prompting strategies but externalized into an auditable service.

E. Service Layer and Governance

The Service Layer provides the infrastructure services required for secure and scalable operation of the system. An authentication server acts as a proxy between the extension and external LLM services, providing:
i. User authentication and session management
ii. API-key management and request routing
iii. Usage logging and monitoring
iv. Rate limiting and cost control
The guardrail service of Section II-D is hosted in this layer, so that the evidence base and thresholds are administered centrally rather than per client.

On top of these services, CodeA11y enforces **governance** as a first-class concern. Each user is associated with budgets—a maximum number of files that may be audited and a maximum number that may be fixed—together with a `stop_on_quota_exceed` policy. As the extension performs audits and applies fixes, it reports consumption to the server, which maintains per-user counters of files audited and files fixed and derives a governance summary: remaining budget, whether each limit has been reached or exceeded, the amount of any overage, and human-readable alert messages. When the stop-on-exceed policy is active and a budget is surpassed, the server signals the client to halt further automated activity. This converts cost and blast-radius control from an informal convention into an enforceable, server-side guarantee—important when LLM-assisted changes are applied across enterprise-scale code-bases. An administrative dashboard allows system administrators to monitor usage statistics, system performance, analysis activity, and per-user governance state.

F. Data and Evidence Layer

The Data Layer maintains persistent storage for analysis results and metadata. Accessibility findings generated by Worker Agents are stored as structured JSON records, allowing the system to maintain a complete audit trail of detected issues and recommended fixes. These records also enable incremental analysis, where previously analyzed bundles—and cached runtime-applicability results—can be reused in subsequent scans. User-session data, configuration settings, governance counters, and execution logs are persisted in a relational database (PostgreSQL or SQLite), enabling efficient tracking of system usage and reproducible analysis.

To make compliance defensible by default, CodeA11y normalizes the workspace's local artifacts—the consolidated report, the per-file issue map, kept and ignored fix decisions, regeneration metadata, guardrail verdicts, and runtime-

applicability counts—into a single canonical *project snapshot* that the extension uploads to the server. A project is identified by a privacy-preserving workspace fingerprint (a salted hash of the workspace path), and only workspace-relative paths are transmitted; raw absolute paths and source contents are never uploaded. Each snapshot records, per project and per file: audited vs. remaining files, issue/fix/ignored counts, applicable-check counts, kept/regenerated counts, compliance scores, and the distribution of guardrail flags (accurate/review/ignore/missing). The server persists these snapshots as immutable, versioned rows so that historical state is preserved rather than overwritten, and exposes owner-scoped user dashboards and organization-wide admin dashboards over them. The result is an audit-ready evidence trail: for any project, one can reconstruct what was detected, what the model recommended, how the guardrail judged each recommendation, and which decisions a human ultimately made.

G. Human-in-the-Loop Workflow

CodeA11y implements a human-in-the-loop approach that positions developers as the final decision-makers in the remediation process. The system operates through a structured workflow that interleaves the Generate, Verify, and Evidence stages:

Step 1 — Runtime-grounded detection: CodeA11y runs the applicability preflight (Section II-C) and scans the codebase, identifying accessibility violations against the checks observed to apply to each file.

Step 2 — AI-generated fixes: for each violation, the system generates context-aware remediation code with a developer-oriented explanation and the relevant WCAG criterion.

Step 3 — Automated verification: each fix's rationale is evaluated by the guardrail (Section II-D); accept/review/reject verdicts and reasons are attached to the proposed edits.

Step 4 — Developer review: developers review proposed fixes alongside their guardrail verdicts and runtime evidence (including artifact screenshots), evaluate appropriateness, and may edit or regenerate a suggestion before application; review queues prioritize flagged and rejected items.

Step 5 — Selective application: developers approve fixes individually or in batches under the active governance budget, maintaining full control over code changes.

Step 6 — Validation and evidence: after applying fixes, CodeA11y re-validates the code to confirm that violations are resolved, and persists the resulting project snapshot (Section II-F), logging every detection, fix, verdict, and decision.

This workflow ensures that AI-generated fixes align with project requirements, coding standards, and organizational policies—verified by the guardrail, grounded in runtime behavior, bounded by governance, and recorded as evidence—while accelerating remediation compared to fully manual approaches. Crucially, humans remain in control of every applied change.

Issue Category	Number of scenarios
Static Violations	106
Structural Violations	61
Behavioral Violations	33

Fig. 2. Test Case Distribution

III. Evaluating CodeA11y

The test-data corpus included common UI elements and workflows such as forms, navigation menus, buttons, images, modals, tables, drag-and-drop, and dynamically updated content. To enable controlled evaluation with known ground truth, we constructed these test cases by seeding WCAG 2.0/2.1/2.2 Level A and AA defects using predefined templates. The defects covered representative failure modes, including missing or incorrect ARIA attributes, poor semantic structure, broken keyboard interaction, insufficient labelling, and screen-reader interoperability issues (e.g., missing announcements or incorrect accessible names). Each seeded defect was logged during corpus construction and used as ground truth when computing identification and remediation coverage. Cumulatively, the corpus comprises 150 SME-designed test scenarios (Figure 2), with accessibility violations organized into three categories:

- **Static Violations:** detectable through source-level analysis, such as missing alternative text, invalid or missing ARIA attributes, color-contrast violations, and improper form labels.
- **Structural Violations:** arising from incorrect semantic organization of the DOM, including broken heading hierarchies, invalid landmark roles, improper element nesting, and missing relationships between labels and controls.
- **Behavioral Violations:** affecting runtime interaction and assistive-technology behavior, such as incorrect keyboard focus order, keyboard traps, inaccessible dynamic updates, and inconsistent screen-reader announcements. These violations are typically difficult to detect using static rules alone and require contextual understanding of the interaction flow; they are the principal target of the runtime applicability layer.

A. Deployment Setup

CodeA11y was integrated into a sandboxed test environment simulating a developer workflow. For each input project (i.e., a multi-file web codebase) the tool executed the following steps:

Step 1 — Runtime applicability preflight: render the live application and compute the per-file applicable WCAG checks (Section II-C).

Step 2 — Accessibility issue identification: detect WCAG violations using LLM-guided, applicability-grounded WCAG evaluation.

Step 3 — Explanation generation: provide a human-readable explanation of each detected issue, including the relevant

Issue Category	WCAG Criteria Evaluated	Detected by CodeA11y	Scenarios Tested	Issue Identified	Coverage (%)
Static Violations	24	23	106	105	99.06
Structural violations	14	11	61	57	93.44
Behavioral Violation	21	14	33	25	75.76

Fig. 3. CodeA11y performance on violations identification

Issue Category	WCAG Criteria Evaluated	Detected by CodeA11y	Scenarios Tested	Issues Remediated	Coverage (%)
Static Violations	24	18	106	104	98.11
Structural violations	14	9	61	55	90.16
Behavioral Violation	21	12	33	22	66.67

Fig. 4. CodeA11y performance on violations remediation

WCAG criterion.

Step 4 — Output verification: score each fix rationale with the guardrail and attach accept/review/reject verdicts.

Step 5 — Automated remediation: generate context-aware, code-level fixes tailored to the specific issue and to the HTML structure, and present before/after diffs for developer review and approval.

Step 6 — Post-fix validation: upon the developer's approval, fixes are applied and re-validated to verify whether the identified accessibility violations were resolved.

All intermediate outputs—detected violations, applicability contexts, guardrail verdicts, proposed fixes, and final code states—were logged for analysis. The backend logs of CodeA11y, capturing every thought, tool call, and state transition, provided the granular data necessary to analyze not just the final code quality, but the process by which the agent arrived at the solution.

B. Result

We measured performance across two primary dimensions: (i) coverage of accessibility-issue identification, and (ii) coverage of accessibility-issue remediation, over the benchmark corpus of 150 accessibility test scenarios annotated against WCAG 2.0, 2.1, and 2.2 Level A and AA SCs. Accuracies are calculated against SME-validated ground truths. Overall, the findings in Figure 3 and Figure 4 suggest that CodeA11y is highly effective for static and structural accessibility-issue identification, while behavioral violations remain more challenging due to their dependence on dynamic interaction contexts, run-time event handling, and assistive-technology feedback, which are harder to correct reliably using source-level edits alone. The runtime applicability layer directly addresses the detection side of this gap by grounding the audit in observed behavior, while the guardrail improves the trustworthiness of remediation by suppressing hallucinated or misattributed rationales before they reach the developer.

IV. Conclusion

By making accessibility continuous, code-first, runtime-grounded, and evidence-backed, CodeA11y enables teams to move from "test and fix later" to *Accessible by Design*—where accessibility is delivered with the same rigor, automation, and governance as security in modern DevOps pipelines. Crucially, humans remain in control: the guardrail provides an evidence-grounded second opinion on every AI rationale, governance bounds the cost and blast radius of automated change, and an audit-ready evidence trail makes compliance defensible by default. Our findings indicate that LLM-assisted remediation can significantly reduce manual accessibility effort by generating context-aware code edits and developer-oriented explanations. The observed gap between identification and remediation, particularly for behavioral violations, highlights the limitations of purely source-level analysis in addressing interaction-driven accessibility concerns; the runtime applicability, output-verification, and governance extensions presented here are our principal response to that gap. Future work focuses on deeper behavioral remediation through richer browser instrumentation and assistive-technology simulation, on closing the loop between guardrail verdicts and automatic fix regeneration, and on extending governed, evidence-backed remediation across the remaining phases of the SDLC.

References

[1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of ai on developer productivity: Evidence from github copilot," *arXiv preprint arXiv:2302.06590*, 2023.

[2] M. Andruccioli, B. Bassi, G. Delnevo, and P. Salomoni, "Leveraging large language models for sustainable and inclusive web accessibility," *Big Data and Cognitive Computing*, vol. 9, no. 10, p. 247, 2025.

[3] J.-M. López-Gil and J. Pereira, "Turning manual web accessibility success criteria into automatic: an llm-based approach," *Universal Access in the Information Society*, vol. 24, no. 1, pp. 837–852, 2025.

[4] A. Othman, A. Dhouib, and A. Nasser Al Jabor, "Fostering websites accessibility: A case study on the use of the large language models chatgpt for automatic remediation," in *Proceedings of the 16th international conference on pervasive technologies related to assistive environments*, 2023, pp. 707–713.

[5] A. M. Alghamdi, W. Aljedaani, H. Jalali, S. Ludi, and M. M. Eler, "Understanding developer challenges and trends in web accessibility: a stack overflow analysis," *Universal Access in the Information Society*, vol. 24, no. 2, pp. 1701–1717, 2025.

[6] L. Nie, H. Liu, J. Sun, K. S. Said, S. Hong, L. Xue, Z. Wei, Y. Zhao, and M. Li, "Sok: Detection and repair of accessibility issues," *arXiv preprint arXiv:2411.19727*, 2024.

[7] A. Bai, H. C. Mork, and V. Stray, "A cost-benefit analysis of accessibility testing in agile software development: results from a multiple case study," *International Journal on Advances in Software*, vol. 10, no. 1&2, pp. 96–107, 2017.

[8] G. Delnevo, M. Andruccioli, and S. Mirri, "On the interaction with large language models for web accessibility: Implications and challenges," in *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*. IEEE, 2024, pp. 1–6.

[9] K. S. Fuglerud, T. Halbach, I. Utseth, and A. U. Waldeland, "Exploring the use of ai for enhanced accessibility testing of web solutions," in *Universal Design 2024: Shaping a Sustainable, Equitable and Resilient Future for All*. IOS Press, 2024, pp. 453–460.

[10] M. Holmlund, "Evaluating chatgpt's effectiveness in web accessibility for the visually impaired," 2024.

[11] C. Huang, A. Ma, S. Vyasamudri, E. Puype, S. Kamal, J. B. Garcia, S. Cheema, and M. Lutz, "Access: Prompt engineering for automated web accessibility violation corrections," *arXiv preprint arXiv:2401.16450*, 2024.

[12] N. Fathallah, D. Hernández, and S. Staab, "Accessguru: Leveraging llms to detect and correct web accessibility violations in html code," in *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*, 2025, pp. 1–22.